

大培桌面端工程架构与技术选型决策文档

基于 ACP 协议驱动的下一代 AI 协作协同空间架构

文件状态: 正式批准 (v1.0)

编写时间: 2026年06月

架构体系: Electron + Monorepo + ACP

核心原则: 重引擎、强契约、轻视图

1. 背景与核心目标

大培 (dapei.skill) 当前已具备完善的 **Skill + 确定性引擎** 形态。底层引擎通过 `runCapability(id, input, ctx)` 执行对工作空间 (workspace)、仓库 (repos)、特征功能 (feature) 及数据契约 (CDR) 的核心操纵，并将知识资产沉淀在磁盘契约目录中。现阶段的核心痛点在于：如何将上述黑盒契约可视化、降低终端用户的上手成本，并以 **Agent-Share (智能体共享)** 模式深度串联 OpenCode、Claude Code 等底层 AI 能力。

1.1 架构演进哲学：从终端模拟到协议驱动

在传统的桌面端 AI 工具设计中，通常使用 PTY (伪终端) 技术拉起 CLI 进程，并通过前端终端组件 (如 xterm.js) 进行文本透传，或通过复杂的正则表达式在主进程流式匹配文本 (即经典的 Emacs comint-mode 拦截模式)。此模式在面对极其复杂的工具调用 (Tool Call) 与状态同步时，会产生严重的乱码截断、高昂的正则解析开销以及脆弱的进程管控。

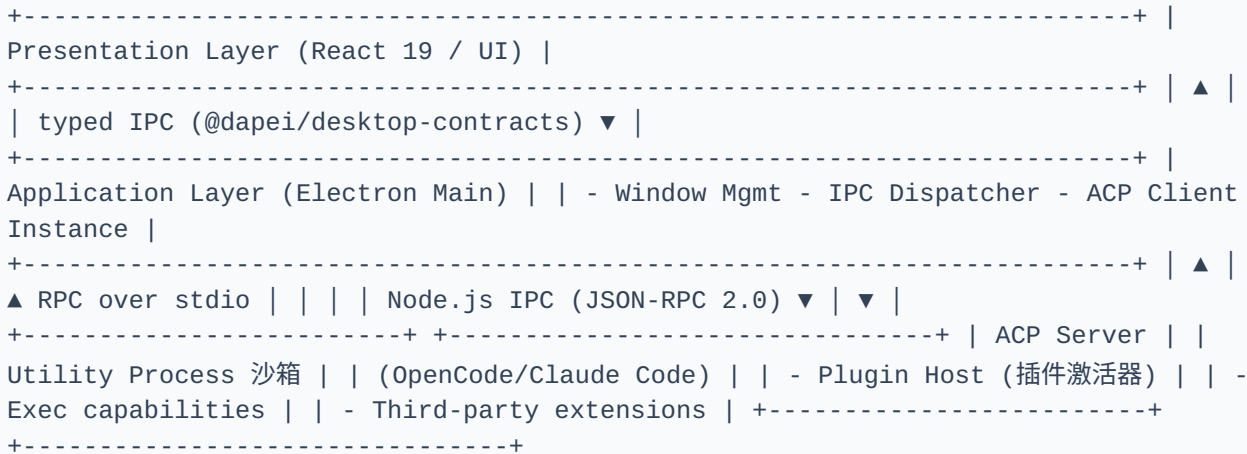
关键决策：大培桌面端架构放弃传统的 PTY 正则硬提取路线，全面对齐并采用现代编辑器标准——**ACP (Agent Client Protocol, 智能体客户端协议)**。将底层引擎和 OpenCode 视为结构化的 JSON-RPC Server，通过标准输入输出 (stdio) 实现强类型、事件驱动的双向通讯，彻底打破 AI 逻辑与桌面表现层的耦合。

2. 核心技术栈选型矩阵

选型维度	指定选型	架构层考量与技术收益
桌面壳容器	Electron 33+	提供成熟的跨平台系统级 API 控制（文件系统级监听、原生进程管控），且完全契合 stdio 管道控制模型。
工程构建工具	electron-vite	开箱即用的主进程（Main）、预加载脚本（Preload）与渲染进程（Renderer）三端双向编译流，原生对 Zod、TypeScript 引用支持极佳。
视图层框架	React 19 + React Router 7	利用 React 19 稳定的并发特性和流式渲染能力，完美对应 ACP 协议的异步 Token 增量推送流。
样式与组件库	Tailwind CSS 4 + shadcn/ui	基于 Tailwind v4 高性能编译；shadcn/ui 具备完全的代码级所有权，便于高度定制大培专有的富文本 Agent 交互卡片。
状态管理分层	TanStack Query + Zustand	Query 负责接管和缓存来自主进程的 ACP “服务端”数据流，带自动失效机制；Zustand 控制轻量级局部 UI 瞬态。
契约与数据校验	Zod	用于统一声明全仓 IPC Payload 与 ACP 协议 Manifest 的强类型 Schema，保证两端数据的健壮性。

3. 进程拓扑与通讯模型

桌面端严格遵循 Electron 的安全设计红线，同时针对 ACP 协议特性进行了进程模型特化。主进程（Main）不再承担高危代码执行，仅作为 ACP 协议的调度中心与管道管理器。



3.1 ACP Client 通讯控制链路

由于放弃了 PTY，主进程通过 Node.js 原生的 `child_process.spawn` 启动底层引擎，建立受控的 `stdin / stdout` 管道。所有通讯基于 JSON-RPC 2.0 标准执行。当底层进行逻辑推理或工具调用时，数据全部以高度结构化的事件投递，完全避免了传统黑盒终端的乱码现象。

3.2 插件沙箱隔离机制 (Utility Process)

针对 Phase M2/M3 开放的第三方插件系统，主进程严禁直接使用动态 `import()` 加载未受信任的第三方代码。系统统一通过 Electron 专属的 **Utility Process** 派生出独立的 Node.js 进程来托管 `PluginHost`。插件如果发生内存泄漏、死循环或异常崩溃，其影响将被完全阻断在沙箱内部，不会导致整个大培桌面主应用闪退。

4. 核心决策深入解析

4.1 决策一：引入 ACP 标准协议，抛弃终端文本硬提取

传统方案通过捕获终端字符拼装行数据，再用正则去“猜”大模型的意图（如判断是否在调用 `Git`）。ACP 协议则将底层彻底抽象为 RPC 服务，通过预定义的 `Method` 与 `Notification` 实现严谨交互。

对比分析：MCP (模型上下文协议) vs ACP (智能体客户端协议)

维度	MCP (Model Context Protocol)	ACP (Agent Client Protocol)	大培桌面端架构抉择结论
定位核心	连接大模型与外部数据源/上下文	专门规范代码编辑器与 Agent 的交互	大培桌面聚焦于工程研发协作，ACP 契合度完美。
流式交互	面向粗粒度的 Prompt 内容组装	原生支持 Token 级别的 Delta 增量流	大培前端需要极其平滑、低延迟的消息气泡渲染。
传输通道	偏向于网络 HTTP / SSE 架构	完全基于受控的 stdio 管道传输	进程级强管控，无需暴露本地网络端口，安全性高。

4.2 决策二：状态管理与主进程推送的精准对接

由于大培桌面端的数据本源存在于底层引擎和磁盘契约中，渲染进程本质上是一个“瘦客户端”。

- **TanStack Query 接管“远端状态”**：诸如工作空间校验结果、Feature 列表、流水线运行状态等，全部视为服务端数据。通过自定义 Preload Hook 封装 IPC 请求，配合自动缓存。
- **事件驱动的局部缓存失效机制**：当底层 ACP Server 产生数据变更（例如执行了 `feature/create`），主进程会捕获到 ACP 事件并向前端广播 `dapei:workspace:mutated`。前端监听该事件后，静默触发 `queryClient.invalidateQueries({ queryKey: ['features'] })`，实现局部精细重绘，完全避免了全局状态树的繁琐流转。

4.3 决策三：基于安全白名单的“人工干预闸门” (Human-in-the-loop)

为防止底层的 AI Agent 在自动执行任务时发生非预期的文件破坏（如误删核心代码库），主进程在 ACP Client 层建立了严格的工具拦截机制。

红线拦截机制：当前端或底层 Agent 发起属于危险 Allowlist 之外的能力调用（如 `repos.remove` 或 `feature.close`）时，ACP Client 会中断当前 stdio 通道的执行流，向渲染层抛出 `agent/toolCallRequest`。渲染层立刻弹出模态“二次确认卡片”并锁定用户输入框。只有当用户手动点击“准许”并校验专属 Token 后，主进程才向 stdio 写入 `toolCallResult` 允许底层继续向下执行。

5. 关键协议与数据流示例

5.1 初始化协商阶段 (initialize)

应用启动挂载工作空间时，两端通过标准格式协商各自的能力边界：

```
// Client (桌面主进程) -> Server (底层引擎)
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "clientInfo": { "name": "dapei-desktop", "version": "1.0.0" },
    "workspaceRoot": "/users/workspace/dapei-project",
    "capabilities": {
      "ui": { "supportsRichCards": true, "supportsModals": true }
    }
  }
}

// Server -> Client 响应
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "capabilities": {
      "tools": ["workspace.validate", "feature.create", "repos.sync"],
      "supportedModels": ["opencode-v2", "claude-3-5-sonnet"]
    }
  }
}
```

5.2 流式对话与工具拦截复合数据流

下述示例展示了 Agent 在流式输出文本时，突然触发能力调用，被桌面端完美拦截并动态渲染的完整事件流：

```

// 1. Server 持续向 Client 推送 Token 流 (UI层实时渲染文字增量)
{"jsonrpc": "2.0", "method": "agent/textStream", "params": { "delta": "正在为您分析当前组件代码..." }}
{"jsonrpc": "2.0", "method": "agent/textStream", "params": { "delta": ", 发现需要同步最新的代码库。" }}

// 2. Server 发现需要执行写操作, 发出工具调用请求
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "agent/toolCallRequest",
  "params": {
    "capabilityId": "repos.sync",
    "input": { "repoName": "dapei-core-runtime", "force": false }
  }
}

// [主进程拦截该请求, 并不下发给终端, 转而向前端 Renderer 投递通知]
// [Renderer 弹出富 UI 确认卡片, 用户点击“同意同步”]

// 3. 桌面端收集用户准入凭证, 将结果回喂给底层 stdio, Agent 恢复运行
{
  "jsonrpc": "2.0",
  "id": 42,
  "result": {
    "approved": true,
    "status": "success",
    "data": { "synchronizedCommits": 14 }
  }
}

```

6. 分期落地工程规划 (Roadmap)

- **Phase M0 — 骨架协议期 (2-3 周):** 搭建 pnpm workspace Monorepo 体系; 建立 `@dapei/desktop-contracts`; 实现主进程利用 `child_process` 挂载标准的 ACP 基础通信管道 (完成 `initialize` 与基础查询)。视图层基于本地静态 HTTP Server 通过 `WebContentView` 完成产品知识 (Knowledge Portal) 内嵌。
- **Phase M1 — 核心交互期 (3-4 周):** 落地 `@dapei/desktop-agent` 的核心事件分发体系; 实现 `agent/textStream` 增量流渲染与 `toolCallRequest` 的前端卡片式二次确认闸门。彻底跑通首个业务闭环: 创建 Feature -> 触发 Agent 思考 -> 拦截工具执行。

- **Phase M2 — 生态与能力治理期 (3-4 周):** 引入 **Utility Process** 插件隔离宿主；实现 L1 UI 路由插件与 L2 AgentBackend 动态注册。强化 `desktop-git` 组件，为 UI 提供全量只读的可视化 Git 观测视图模型。
- **Phase M3 — 跨智能体全景协同 (长期):** 开放多 Agent 并行 Session 调度 (ACP Bridge) ；集成完善的证据链追溯视图与 CDR 流水线高级监控面板。